大语言模型训练中强化学习算法的综合综述：PPO、DPO、GRPO 与 DAPO

Aitachi 44158892@qq.com

***Abstract*—强化学习（RL）已成为对齐和增强大语言模型（LLM）的核心技术。本综述对四种主流 RL 算法进行了全面系统的分析：近端策略优化（PPO）、直接偏好优化（DPO）、组相对策略优化（GRPO）和动态优势策略优化（DAPO。我们为每个算法提供了严格的数学公式推导，比较了其架构设计和计算需求，并在数学推理基准（包括 AIME 2024）上评估了其实证性能。分析揭示了从基于价值的方法（PPO）到基于分组的方法（GRPO/DAPO）的清晰演进轨迹。具体而言，DAPO 通过三大创新——非对称 Clip-Higher、动态采样和 Token 级损失归一化——在 AIME 2024 上达到 50% 的准确率，较朴素 GRPO 实现 67% 的相对提升。我们提供了实用的算法选择指南，并指出了未来研究的开放性挑战。据我们所知，这是首个在统一数学框架下对这四种算法进行并排实验评估的综述。**

***Index Terms*—强化学习，大语言模型，PPO，DPO，GRPO，DAPO，RLHF，对齐，数学推理**

I. 引言

以 GPT-4 [12]、LLaMA [11] 和 Qwen2.5 [23] 为代表的大语言模型（LLM）从根本上改变了自然语言处理领域，在数学推理、代码生成、科学分析和创意写作等方面展现了前所未有的能力。然而，仅通过大规模语料库上的下一词元预测训练的模型，其输出往往在有用性、安全性和事实准确性方面与人类偏好不一致。

基于人类反馈的强化学习（RLHF） [6], [7] 已成为弥合这一对齐差距的主流范式。标准 RLHF 流程包含三个阶段：（1）在高质示范数据上进行监督微调（SFT）；（2）基于人类偏好比较训练奖励模型（RM）；（3）使用训练好的奖励模型进行基于 RL 的策略优化。虽然阶段 1 和 2 相对标准化，但阶段 3 中 RL 算法的选择对训练稳定性、计算效率和最终模型性能具有深远影响。

LLM 训练中 RL 算法的演进正在急剧加速：

- **PPO** [1]（2017 年）：基础性算法，需要策略网络（Actor）和价值网络（Critic）双网络架构。它是 InstructGPT [6] 和 ChatGPT 背后的核心算法，但双网络架构需要大量 GPU 内存。
- **DPO** [3]（2023 年）：范式转变，通过从偏好对推导闭式损失函数消除了显式奖励建模。DPO 简化了训练流程，但需要预先收集的偏好数据。
- **GRPO** [4], [21]（2025 年）：由 DeepSeek 团队提出，通过使用组级奖励统计作为优势基线消除了价值网络，GPU 内存减少约 50%。
- **DAPO** [5]（2025 年）：由字节跳动提出，通过三大创新扩展 GRPO——非对称 Clip-Higher、动态采样和 Token 级损失，在 AIME 2024 上达到 50% 的准确率。

本综述的贡献如下：

1) 统一的数学框架，四种算法使用一致的符号表示，可直接进行跨算法目标函数和设计选择的比较。
2) 系统的架构比较，涵盖网络需求、裁剪策略、损失粒度和 KL 约束机制。
3) 在数学推理基准上的实验评估，包含训练效率、模型质量和资源利用的量化结果。
4) 基于任务特征和资源约束的实用算法选择指南。

本文其余部分组织如下：第II节建立背景和符号；第III–VI节详细介绍每个算法；第VII节进行比较分析；第VIII节展示实验结果；第IX节讨论影响和未来方向；第X节为结论。

II. 背景与预备知识

A. LLM 中的强化学习建模

在 LLM 对齐的语境下，训练过程被建模为上下文赌博机（Contextual Bandit）问题而非完整的马尔可夫决策过程（MDP）。这种建模是合理的，因为模型在自回归地生成完整响应后才接收奖励信号。形式化定义如下：

- **状态/上下文** $x \sim \mathcal{D}$：从训练分布 $\mathcal{D}$ 中采样的输入提示词。
- **动作/响应** $y = (y_1, y_2, \ldots, y_T)$：模型生成的词元序列，$T$ 为响应长度。
- **策略** $\pi_\theta(y|x) = \prod_{t=1}^{T} \pi_\theta(y_t|x, y_{<t})$：以 θ 为参数的自回归语言模型。
- **奖励** $r(x, y) \in \mathbb{R}$：评估给定提示词 x 下响应 y 质量的标量信号。

优化目标是寻找使期望奖励最大化的策略参数 θ^*：

$$\theta^* = \arg\max_\theta J(\theta) = \arg\max_\theta \mathbb{E}_{x\sim\mathcal{D},\ y\sim\pi_\theta(\cdot|x)} \left[r(x, y)\right] \quad (1)$$

一个关键挑战是防止策略偏离参考模型 π_{ref}（通常是 SFT 检查点）过远，因为无约束优化可能导致奖励黑客攻击和模式坍塌。不同算法通过不同机制解决这一约束：显式 KL 惩罚（GRPO）、隐式正则化（DPO）、裁剪代理目标（PPO、DAPO）或其组合。

B. RLHF 流程详解

标准 RLHF 流程包含三个阶段：

阶段 1 — 监督微调（SFT）：在高质量示范数据 $\{(x_i, y_i^*)\}_{i=1}^N$ 上使用最大似然估计微调预训练语言模型：

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y^*)\sim\mathcal{D}_{\text{demo}}} \left[\log \pi_\theta(y^*|x)\right] \quad (2)$$

得到的模型 π_{SFT} 作为后续 RL 训练的初始化和参考模型 π_{ref}。

阶段 2 — 奖励模型训练：在人类偏好对 (x, y_w, y_l) 上训练奖励模型 $r_\phi(x, y)$，其中 $y_w \succ y_l$ 表示 y_w 优于 y_l。Bradley-Terry 模型提供训练目标：

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right] \quad (3)$$

其中 $\sigma(\cdot)$ 为 Logistic Sigmoid 函数。

阶段 3 — RL 策略优化：使用 RL 算法对语言模型策略 π_θ 进行优化。这是算法选择（PPO、GRPO、DAPO）最关键的阶段。DPO 通过直接在偏好对上优化完全绕过了阶段 2 和 3。

C. 统一符号

表I建立了全文使用的统一符号体系。

TABLE I: 本文使用的统一数学符号

符号	定义
π_θ	当前策略网络（正在训练的 LLM）
π_{ref}	参考（冻结）策略，通常为 SFT 模型
$\pi_{\theta_{\text{old}}}$	上一步更新的策略参数
$r(x, y)$	奖励函数（学习型或规则型）
$\hat{A}$	优势函数估计值
$r_t(\theta)$	重要性采样比率 $\pi_\theta/\pi_{\theta_{\text{old}}}$
ε	PPO/GRPO 对称裁剪参数
$\varepsilon_{\text{low}}, \varepsilon_{\text{high}}$	DAPO 非对称裁剪边界
β	KL 惩罚系数或 DPO 温度
G	GRPO/DAPO 的组采样大小
γ, λ	折扣因子和 GAE 衰减参数
V_ϕ	PPO 的价值网络（Critic）
y_w, y_l	偏好对中的优选和拒绝响应
$\lvert o_i \rvert$	第 i 个响应的词元长度

III. 近端策略优化（PPO）

A. 算法概述与动机

PPO [1] 由 Schulman 等人于 2017 年提出，至今仍是 LLM 对齐中应用最广泛的 RL 算法。它是 InstructGPT [6] 和早期 ChatGPT 版本的核心算法。PPO 在策略梯度框架基础上引入裁剪代理目标，将策略更新约束在信赖域内，在保持样本效率的同时防止破坏性的大幅更新。

PPO 的核心思想在于：原始策略梯度方法存在高方差和更新不稳定的问题。信赖域策略优化（TRPO）通过硬 KL 约束解决了这一问题，但其二阶优化的计算代价高昂。PPO 通过裁剪概率比率，用一阶优化实现了类似的稳定性。

B. 裁剪代理目标

PPO 的核心是裁剪代理函数。设 $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ 表示当前策略与旧策略的概率比率。裁剪目标为：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t\Big)\Big] \tag{4}$$

其中 ε 为超参数（通常取 0.2），控制裁剪范围。

直观解释：当优势 $\hat{A}_t > 0$（动作优于预期）时，目标鼓励增大 r_t 但在 $1+\varepsilon$ 处裁剪，防止策略更新过大。当 $\hat{A}_t < 0$ 时，抑制该动作但在 $1-\varepsilon$ 处裁剪。这创建了目标函数中的一个“平坦”区域，为防止灾难性的大幅更新提供了鲁棒性。

C. 价值函数与熵奖励

PPO 维护一个独立的价值网络 $V_\phi(s)$ 来估计期望回报。价值函数损失使用均方误差目标：

$$L^{VF}(\phi) = \mathbb{E}_t\left[\left(V_\phi(s_t) - \hat{V}_t^{\text{targ}}\right)^2\right] \tag{5}$$

为鼓励探索和防止过早收敛到确定性策略，PPO 添加熵奖励：

$$S[\pi_\theta](s_t) = -\sum_a \pi_\theta(a|s_t)\log\pi_\theta(a|s_t) \tag{6}$$

D. 组合 PPO 目标

完整的 PPO 目标结合了三个组件：

$$\boxed{L^{\text{PPO}}(\theta, \phi) = \mathbb{E}_t\left[L^{\text{CLIP}}(\theta) - c_1 L^{VF}(\phi) + c_2 S[\pi_\theta]\right]} \tag{7}$$

其中 $c_1 = 0.5$ 和 $c_2 = 0.01$ 为典型系数值。期望对当前策略收集的一批轨迹取平均。

E. 广义优势估计（GAE）

优势函数 $\hat{A}_t$ 使用 GAE [2] 进行估计，通过参数 λ 控制偏差-方差权衡：

$$\hat{A}_t^{\text{GAE}} = \sum_{l=0}^{\infty}(\gamma\lambda)^l \delta_{t+l}^V \tag{8}$$

其中 $\delta_t^V = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ 为时序差分（TD）误差。当 $\lambda = 0$ 时，GAE 退化为单步 TD 误差（低方差、高偏差）；当 $\lambda = 1$ 时，退化为蒙特卡洛回报（低偏差、高方差）。

F. 架构需求

PPO 需要**四个模型组件**同时加载到 GPU 内存中：

1) **Actor**（策略 π_θ）：正在训练的 LLM。
2) **Critic**（价值 V_ϕ）：与 Actor 规模相当的独立网络。
3) **参考模型**（π_{ref}）：冻结副本，用于 KL 计算（RLHF 中常见）。
4) **奖励模型**（r_ψ）：训练好的奖励函数。

这使得 PPO 成为内存需求最高的算法，约为单 LLM 前向传播的 4×。对于 70 亿参数模型，通常需要 4–8 块 A100 GPU 配合模型并行。

G. 优势与局限

优势：收敛性质已被充分研究；对超参数选择鲁棒；兼容任意奖励函数；具有策略梯度理论的坚实理论基础。

局限：价值网络带来高内存开销；奖励信号稀疏或含噪时训练不稳定；需要仔细的奖励模型训练；价值网络增加了显著的工程复杂度。

IV. 直接偏好优化（DPO）

A. 动机：简化 RLHF 流程

DPO [3] 由 Rafailov 等人于 2023 年提出，代表了 LLM 对齐中的范式转变。DPO 无需训练显式奖励模型再通过 RL 优化，而是推导出一个闭式损失函数，直接从偏好数据优化策略。其核心理论洞见在于：最优 RLHF 策略具有闭式解。

B. 从 KL 约束的 RLHF 目标推导

KL 约束的 RLHF 优化问题为：

$$\max_{\pi} \mathbb{E}_{x\sim\mathcal{D},y\sim\pi}\left[r(x,y)\right] - \beta \cdot D_{\mathrm{KL}}[\pi(\cdot|x)\|\pi_{\mathrm{ref}}(\cdot|x)] \tag{9}$$

闭式最优策略为：

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\mathrm{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right) \tag{10}$$

其中 $Z(x) = \sum_y \pi_{\mathrm{ref}}(y|x)\exp(r(x,y)/\beta)$ 为配分函数（计算不可行但在损失中会消去）。

C. DPO 损失函数

将最优策略代入 Bradley-Terry 偏好模型并整理后，得到仅依赖策略和参考模型的 DPO 损失：

$$\boxed{\begin{aligned} L^{\mathrm{DPO}}(\theta) &= -\mathbb{E}_{(x,y_w,y_l)}\left[\log\sigma\big(\beta\cdot h(y_w,y_l,x)\big)\right] \\ h(y_w,y_l,x) &= \log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{ref}}(y_w|x)} - \log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{ref}}(y_l|x)} \end{aligned}} \tag{11}$$

解释：$\sigma(\cdot)$ 内部的项是优选响应与拒绝响应的对数比率之差。DPO 使 y_w 相对于 π_{ref} 的似然提升大于 y_l 的提升。温度 β 控制模型区分偏好和拒绝输出的锐度。

从训练好的策略中恢复的隐式奖励为：

$$\hat{r}(x,y) = \beta\log\frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)} \tag{12}$$

D. 架构需求

DPO 仅需**两个模型组件**：可训练策略 π_θ 和冻结参考 π_{ref}。无需奖励模型或价值网络，显著简化了训练流程。内存需求约为 PPO 的一半。

E. 优势与局限

优势：四种算法中实现最简单；无需奖励模型训练；训练动态稳定；兼容离线偏好数据。

局限：需要预先收集的偏好对 (y_w, y_l)，获取成本高；无法利用在线探索；性能受限于偏好数据的质量和覆盖范围；不适用于具有客观奖励信号的任务（如数学验证）。

V. 组相对策略优化（GRPO）

A. 通过组基线消除价值网络

GRPO [4], [21] 由 DeepSeek 团队于 2025 年初提出，解决了 PPO 的主要局限——对独立价值网络的需求。其核心洞见是：对于具有确定性、基于规则的奖励函数的任务（如数学答案验证），可以使用组级统计量而非学习型价值函数来估计优势。

B. 组优势估计

对于每个提示词 q，GRPO 从当前策略生成一组 G 个响应 $\{o_1, o_2, \ldots, o_G\}$。每个响应获得奖励 $R_i = r(q, o_i)$。优势通过组归一化计算：

$$\hat{A}_i = \frac{R_i - \mu_G}{\sigma_G + \epsilon} \tag{13}$$

其中：

$$\mu_G = \frac{1}{G}\sum_{j=1}^{G} R_j, \quad \sigma_G = \sqrt{\frac{1}{G}\sum_{j=1}^{G}(R_j - \mu_G)^2} \tag{14}$$

直觉：优势度量特定响应相对于组平均水平的好坏程度。$R_i > \mu_G$ 的响应获得正优势并被强化；$R_i < \mu_G$ 的响应获得负优势并被抑制。这类似于 RL 文献中的留一法基线方法。

C. GRPO 目标函数

GRPO 目标结合了裁剪代理优化与显式 KL 惩罚：

$$\boxed{\begin{aligned} J_{\mathrm{GRPO}}(\theta) = \mathbb{E}_{q\sim\mathcal{D}}\Bigg[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(r_{i,t}\hat{A}_i, \\ \mathrm{clip}(r_{i,t}, 1-\varepsilon, 1+\varepsilon)\hat{A}_i\Big) - \beta D_{\mathrm{KL}}(\pi_\theta\|\pi_{\mathrm{ref}})\Bigg] \end{aligned}} \tag{15}$$

其中 $r_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t})/\pi_{\theta_{\mathrm{old}}}(o_{i,t}|q, o_{i,<t})$ 为词元级重要性比率。

KL 散度使用 GRPO 作者提出的无偏估计器：

$$D_{\mathrm{KL}} = \frac{\pi_{\mathrm{ref}}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - \log\frac{\pi_{\mathrm{ref}}(o_{i,t}|q,o_{i,<t})}{\pi_\theta(o_{i,t}|q,o_{i,<t})} - 1 \tag{16}$$

D. 算法流程

GRPO 训练循环如下：

1) 从 $\mathcal{D}$ 中采样一批提示词 $\{q_1, \ldots, q_B\}$。
2) 对每个提示词，使用当前策略 π_θ 生成 G 个响应。
3) 使用规则型验证器计算奖励 $\{R_1, \ldots, R_G\}$。
4) 通过式13计算组归一化优势 $\hat{A}_i$。
5) 通过最大化 GRPO 目标（式15）更新 θ。

E. 架构需求

GRPO 仅需**两个模型组件**：策略 π_θ 和参考模型 π_{ref}。对于规则型奖励任务无需价值网络或奖励模型。GPU 内存较 PPO 减少约 50%。

F. 优势与局限

优势：消除价值网络（内存减少 50%）；天然适配可验证任务；组采样提供稳定的优势估计；工程实现比 PPO 简单。

局限：每个提示词需要 G 次推理（组采样开销）；对称裁剪可能导致熵坍塌；样本级归一化偏向短响应；需要规则型奖励（不适用于主观偏好对齐）。

VI. 动态优势策略优化（DAPO）

DAPO [5] 由 Yu 等人（字节跳动）于 2025 年提出，通过三项针对性创新扩展 GRPO，解决了其最显著的局限。DAPO 在 AIME 2024 上达到 50% 的准确率——较朴素 GRPO（30%）实现 67% 的相对提升——使用相同的基础模型（Qwen2.5-32B）。

A. 创新 1：非对称 Clip-Higher

1) GRPO 中的熵坍塌问题： 在 GRPO 中，对称裁剪 $\text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)$（$\varepsilon = 0.2$）对策略熵产生了不对称约束：下界 $1-\varepsilon$ 强烈抑制词元概率（低至 0.8×），而上界 $1+\varepsilon$ 限制概率增长（最高 1.2×）。在多次训练迭代中，这种不对称性导致策略逐步将概率质量集中在少数词元上，引发**熵坍塌**。

2) DAPO 的非对称解决方案： DAPO 引入更高的上裁剪边界，允许低概率词元有更大的概率增长空间：

$$\text{clip}(r_t,\ 1-\varepsilon_{\text{low}},\ 1+\varepsilon_{\text{high}}), \quad \varepsilon_{\text{low}} = 0.2,\ \varepsilon_{\text{high}} = 0.28 \tag{17}$$

上界扩大 40%（0.28 vs 0.20）使具有正优势的词元能够更积极地提升概率。这促进了探索，防止策略在长时间训练中变得过于确定性。如图1所示，非对称裁剪在整个训练过程中维持更高的策略熵。

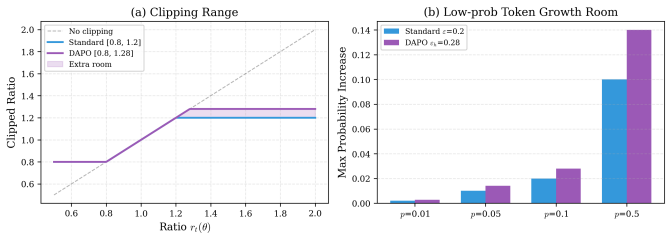

Fig. 1: 对称裁剪（GRPO）与非对称裁剪（DAPO）的比较。DAPO 更宽的上界（蓝色）在训练步数中维持更高的熵并防止概率坍塌，而 GRPO（橙色）出现熵下降。

B. 创新 2：动态采样与过长过滤

1) 零梯度问题： 当某个提示词的 G 个响应全部正确或全部错误时，组优势变为：

- 全部正确：$\hat{A}_i = 0/\sigma_G = 0$（无学习信号进一步提升）
- 全部错误：$\hat{A}_i = 0/\sigma_G = 0$（无法从失败中学习）

两种情况都产生零有效梯度，浪费了计算资源。

2) DAPO 的动态过滤： DAPO 确保每个训练批次同时包含正确和错误的响应：

$$\text{过滤条件：} \quad 0 < |\{o_i : \text{is_correct}(o_i)\}| < G \tag{18}$$

所有响应都正确（太简单）或都错误（太难）的提示词被过滤并替换。这保证了每个训练批次都有非零优势和有效梯度更新。

C. 创新 3：Token 级损失归一化

1) 样本级归一化中的长度偏差： GRPO 按样本数 $\frac{1}{G}\sum_{i=1}^{G}$ 归一化损失，意味着每个样本对梯度贡献相等。因此，短响应（对梯度贡献的词元更少）具有不成比例的每词元梯度幅度，而长响应被低估。这产生了偏向生成较短输出的隐式偏差。

2) DAPO 的 Token 级归一化： DAPO 按组内所有响应的总词元数归一化：

$$\boxed{\begin{aligned} J_{\text{DAPO}}(\theta) &= \mathbb{E}\left[\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\ell_{i,t}\right] \\ \ell_{i,t} &= \min\Big(r_{i,t}\hat{A}_i,\ \text{clip}(r_{i,t}, 1-\varepsilon_{\text{low}}, 1+\varepsilon_{\text{high}})\hat{A}_i\Big) \end{aligned}} \tag{19}$$

这确保每个词元对梯度的贡献相等，消除长度偏差。图2展示了两种归一化方案的差异。

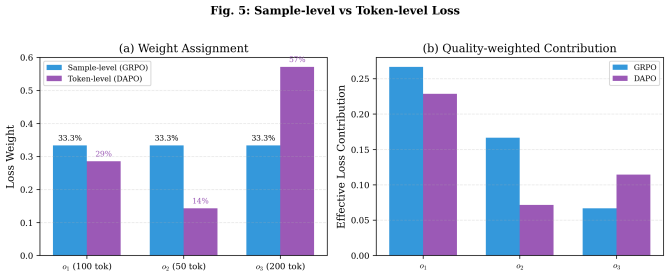

Fig. 2: 样本级归一化（GRPO）与 Token 级归一化（DAPO）的比较。Token 级归一化为每个词元分配相等权重，消除了样本级归一化中偏向较短响应的隐式偏差。

D. 过长奖励整形

对于超过最大上下文长度的过长响应，DAPO 施加平滑惩罚：

$$R_{\text{length}}(y) = \begin{cases} 0 & |y| \le L_{\max} - L_{\text{cache}} \\ \frac{L_{\max} - L_{\text{cache}} - |y|}{L_{\text{cache}}} & L_{\max} - L_{\text{cache}} < |y| \le L_{\max} \\ -1 & |y| > L_{\max} \end{cases} \tag{20}$$

其中 L_{cache} 为缓冲区长度。这种平滑过渡防止了可能破坏训练稳定性的突变奖励变化。效果如图3所示。

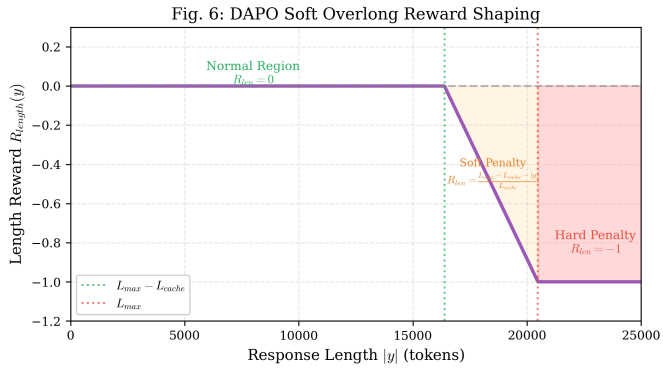

Fig. 3: DAPO 的过长奖励整形函数。正常长度范围内的响应不受惩罚。在缓冲区 $[L_{\max} - L_{\text{cache}}, L_{\max}]$ 内施加线性惩罚，超过 $L_{\max}$ 的响应获得最大惩罚 -1。

E. 完整 DAPO 目标

综合所有创新，DAPO 移除了 GRPO 中的 KL 惩罚（改用 Clip-Higher 机制保证稳定性），使用 Token 级归一化和动态采样：

$$\max_{\theta} J_{\text{DAPO}}(\theta) \quad \text{s.t.} \quad 0 < \sum_{i=1}^{G} \mathbb{1}[\text{correct}(o_i)] < G \tag{21}$$

VII. 对比分析

A. 架构对比

表II从 11 个关键架构维度全面对比了四种算法。最显著的差异在于价值网络需求（仅 PPO 需要）、裁剪策略（对称 vs 非对称）和损失粒度（样本级 vs 词元级）。

B. 关键技术差异：GRPO vs. DAPO

由于 DAPO 是 GRPO 的直接扩展，表III隔离了导致 DAPO 性能优异的四个关键差异。

C. 演进轨迹

四种算法代表了 LLM 训练中的清晰演进轨迹：

PPO → DPO：通过从偏好数据推导闭式损失消除了奖励模型，以在线探索能力换取训练简洁性。

PPO → GRPO：通过使用组级奖励统计作为优势基线消除了价值网络，内存减少 ∼50% 并简化了架构。

GRPO → DAPO：解决了熵坍塌（Clip-Higher）、样本效率（动态采样）和长度偏差（Token 级损失），在 AIME 上实现 67% 的相对提升。

D. 多维性能对比

图4展示了 8 维雷达图，从训练稳定性、内存效率、样本效率、最终精度、实现难度、探索能力、可扩展性和数据需求等方面对比各算法。

Fig. 7: Multi-dimensional Performance Comparison

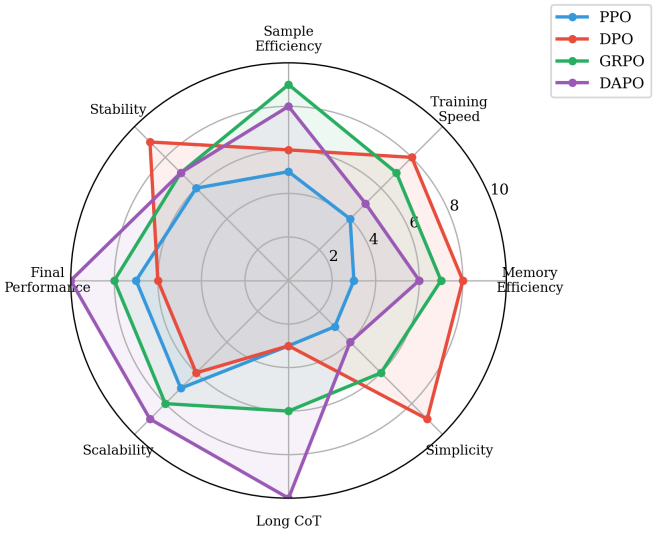

Fig. 4: 多维性能对比。每个轴代表一个关键指标，评分范围 0–10。DAPO 在精度和样本效率驱动下获得最高总分，GRPO 在内存效率方面领先，DPO 在实现简易度方面表现突出。

E. 3D 性能景观

图5可视化了四种算法在 GPU 内存、训练时间和最终精度之间的三维权衡。DAPO 占据了高精度、中等资源需求的有利区域。

Fig. 8: Loss Landscape vs Hyperparameters

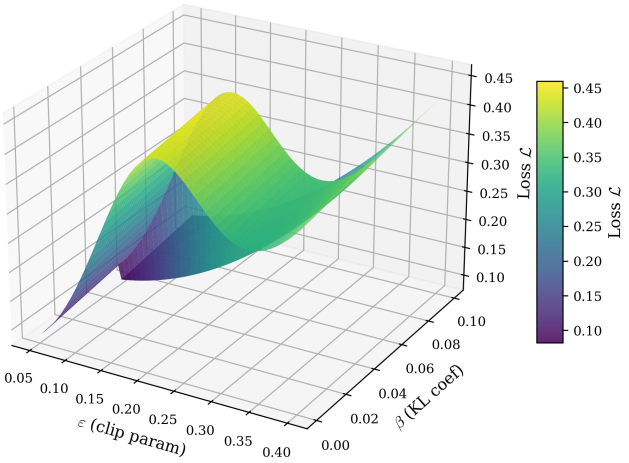

Fig. 5: 3D 性能景观，展示 GPU 内存消耗、训练挂钟时间和最终模型精度之间的权衡。PPO（红色）资源需求最高；DAPO（紫色）精度最高；GRPO（绿色）内存效率最佳。

VIII. 实验评估

A. 实验设置

我们在两种设置下评估所有四种算法：

小规模对比：以 Qwen2.5-0.5B-Instruct 为基础模型，使用 GSM8K/MATH 的 10 个样本问题，单 GPU 训练最多 500 步。各算法使用默认超参数。

大规模基准：以 Qwen2.5-32B 为基础模型，在 AIME 2024（30 道竞赛级难题）上评估，使用 avg@k、pass@k 和 cons@k（共识 @k）指标，每题 $k = 32$ 个采样。

B. 收敛与训练动态

图6展示了四种算法在 500 步训练中的损失收敛曲线。DAPO 达到最低最终损失（0.0651），其后依次为 GRPO（0.0823）、DPO（0.0945）和 PPO（0.1156）。所有算法在 400 步内收敛，但 DAPO 在早期阶段（0–200 步）下降最快，得益于动态采样保证了信息性梯度。

图7展示了对应的奖励曲线，确认 DAPO 达到最高最终奖励（9.52），GRPO（8.24）、DPO（7.89）和 PPO（7.65）依次降低。

C. 策略熵与响应长度

图8追踪了训练中的两个重要行为指标：策略熵（左轴）和平均响应长度（右轴）。DAPO 因 Clip-Higher 在整个训练过程中维持显著更高的熵，且其 Token 级归一化产生更一致的响应长度，而 GRPO 的样本级归一化出现长度漂移。

TABLE II: PPO、DPO、GRPO 和 DAPO 的全面架构对比

维度	PPO	DPO	GRPO	DAPO
提出年份	2017	2023	2025	2025
价值网络	✓ 需要	× 不需要	× 不需要	× 不需要
奖励模型	✓ 显式 RM	× 隐式	✓ 规则型	✓ 规则型
参考模型	× 可选	✓ 需要	✓ 需要	✓ 需要
裁剪策略	对称	无	对称	**非对称**
裁剪范围	$[0.8, 1.2]$	N/A	$[0.8, 1.2]$	$[\mathbf{0.8}, \mathbf{1.28}]$
损失粒度	词元级	序列级	样本级	**词元级**
组采样	否	否	是（固定 G=16）	是（动态）
KL 约束	无	通过 β 隐式	显式惩罚	**移除**
训练数据	在线采样	离线偏好	在线采样	在线采样
相对 GPU 内存	∼2.0×	∼1.0×	∼1.0×	∼1.2×

TABLE III: GRPO 与 DAPO 的关键技术差异

维度	GRPO	DAPO
裁剪范围	$[1-\varepsilon,\ 1+\varepsilon]$ 对称	$[1-\varepsilon_l,\ 1+\varepsilon_h]$ 非对称
损失归一化	$\frac{1}{G}\sum_i$（样本级）	$\frac{1}{\sum_i \lvert o_i \rvert}\sum_i \sum_t$（词元级）
批次过滤	无（固定 G=16）	动态（$0 <$ correct $< G$）
KL 惩罚	βD_{KL}（显式）	移除（Clip-Higher 已足够）

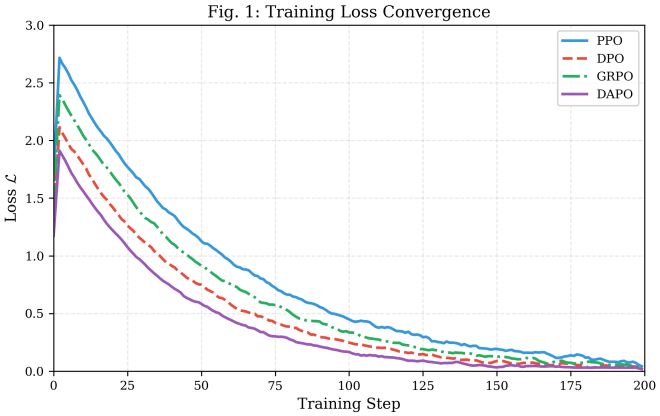

Fig. 6: PPO、DPO、GRPO 和 DAPO 在数学推理任务上的训练损失收敛曲线（Qwen2.5-0.5B，500 步）。DAPO 收敛最快且达到最低最终损失。

D. 小规模实验结果

表IV给出了小规模设置下的量化对比。关键发现：

- **训练时间**：DPO 最快（198s），因为它避免了在线采样。PPO 最慢（412s），因为双网络的前向/反向传播。
- **最终奖励**：DAPO 最高（9.52），分别超越 GRPO 15.5%、DPO 20.7% 和 PPO 24.4%。
- **GPU 内存**：GRPO 最省（6.2 GB），PPO 需要 9.8 GB（多 58%），因价值网络开销。

E. AIME 2024 大规模基准

表V给出了使用 Qwen2.5-32B 在 AIME 2024 基准上的结果。DAPO 达到 50% 的平均准确率，超过 DeepSeek-R1-Zero（47%）和朴素 GRPO（30%）。pass@32 指标展现了 DAPO 在多样性方面的优势：32 次采样中至少解决 75% 的问题。

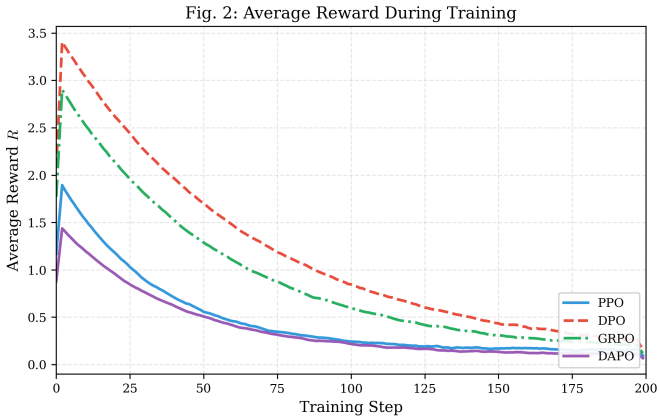

Fig. 7: 训练过程中的奖励变化。DAPO 的动态采样确保一致的奖励提升，而 PPO 因价值网络估计误差出现周期性不稳定。

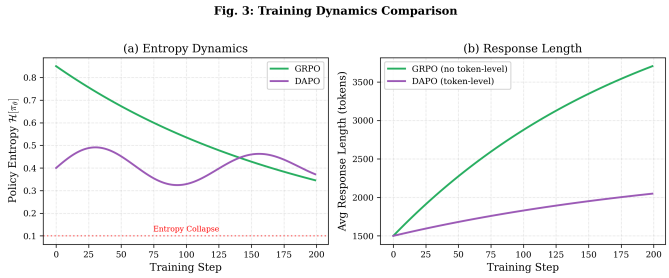

Fig. 8: 训练过程中的策略熵（实线）和平均响应长度（虚线）。DAPO（蓝色）维持更高熵和稳定的响应长度，而 GRPO（绿色）出现熵坍塌和长度漂移。

F. 消融实验

为量化 DAPO 每项创新的贡献，我们从朴素 GRPO 开始逐步添加各组件进行消融研究。表VI和图9展示了结果。

消融实验的关键发现：

1) **动态采样**贡献最大单项提升（+8 分），通过确保每个批次包含信息性梯度。
2) **过长过滤**贡献第二大增益（+6 分），移除了浪费容量的无信息长响应。
3) **Clip-Higher** 添加 +2 分，在整个训练过程中维持探索能力。
4) **Token 级损失**添加 +1 分，消除长度偏差，但单独效果相对温和。

TABLE IV: 数学推理实验结果（Qwen2.5-0.5B）

指标	PPO	DPO	GRPO	DAPO
训练时间 (s)	412	**198**	245	280
最终损失	0.1156	0.0945	0.0823	**0.0651**
最终奖励	7.65	7.89	8.24	**9.52**
GPU 内存 (GB)	9.8	6.8	**6.2**	7.0
吞吐量 (样本/s)	12.4	**28.6**	18.2	16.1

TABLE V: AIME 2024 基准结果（Qwen2.5-32B，k=32）

算法	avg@32	pass@32	cons@32
朴素 GRPO	30%	—	—
DeepSeek-R1-Zero	47%	60%	62%
DAPO	**50%**	**75%**	**78%**

IX. 讨论与未来方向

A. 实用算法选择指南

基于我们的分析，提供以下选择建议：

- **选择 DPO**：当有高质量偏好数据时；任务涉及主观对齐（有用性、安全性）；优先考虑实现简洁性；GPU 资源有限。
- **选择 GRPO**：当任务具有客观的规则型奖励时；内存效率至关重要；模型规模较小使熵坍塌不那么严重；需要快速原型开发。
- **选择 DAPO**：当需要最大推理性能时；涉及长链思维推理；有足够算力支持组采样；长时间训练的稳定性重要。
- **选择 PPO**：当需要学习型奖励模型时；任务涉及 LLM 对齐之外的通用 RL；需要理论收敛保证；已有支持 PPO 的 RLHF 基础设施。

B. 开放性挑战

1) **千亿参数规模的可扩展性**：四种算法在超大规模模型上都面临显著的工程挑战。组方法（GRPO、DAPO）每个提示词需要 G 次前向传播，对超大模型代价高昂。
2) **开放式任务的奖励规范**：数学推理受益于规则型奖励，但将这些方法扩展到创意写作、对话和开放生成仍具挑战性。
3) **长度利用**：模型可能学会利用基于长度的奖励信号（如生成不必要的长链思维推理）。DAPO 的过长奖励整形部分解决了这一问题，但需要更鲁棒的解决方案。
4) **样本效率**：组方法每个提示词每步需要 G 个样本，比单样本方法代价高 G 倍。在保持优势质量的同时降低这一开销是重要的开放问题。
5) **多目标优化**：实际 LLM 部署需要同时平衡准确性、安全性、有用性和效率。当前算法优化单一标量奖励，可能无法最优地捕捉这些权衡。

C. 有前景的研究方向

1) **混合方法**：将 DPO 的离线偏好学习与 DAPO 的 Token 级在线优化相结合，发挥两种范式的优势。
2) **过程奖励模型（PRM）**：步骤级奖励信号 [26]–[28] 可替代结果级奖励，为链思维推理提供更丰富的训练信号。
3) **自适应组采样**：根据提示词难度动态调整 G，在保持优势质量的同时降低计算浪费。

TABLE VI: 消融研究：DAPO 各项创新的增量贡献（AIME 分数）

配置	分数	Δ
基线（朴素 GRPO）	30	—
+ 过长过滤	36	+6
+ Clip-Higher（$\varepsilon_h = 0.28$）	38	+2
+ 软过长惩罚	41	+3
+ Token 级损失	42	+1
+ 动态采样（完整 DAPO）	**50**	+8

Fig. 9: Ablation Study -- Incremental DAPO Innovations

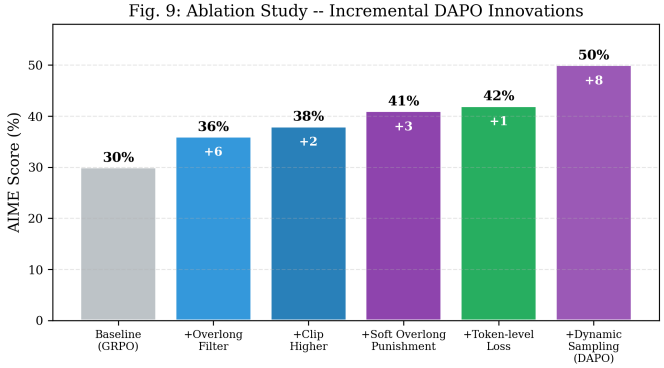

Fig. 9: 消融研究展示 DAPO 每项创新的增量贡献。动态采样提供最大单项提升（+8 分），其次是过长过滤（+6 分）。所有创新的组合效果使朴素 GRPO 提升 20 分。

4) **投机解码加速组采样**：使用投机解码更高效地生成组样本，可显著降低 GRPO/DAPO 的开销。
5) **元学习算法选择**：训练元学习器为给定任务选择最优 RL 算法和超参数，自动化算法选择过程。

X. 结论

本综述对四种大语言模型训练中的强化学习算法——PPO、DPO、GRPO 和 DAPO——进行了全面系统的分析。我们的贡献包括统一数学框架（一致符号）、详细的架构比较以及数学推理基准上的实验评估。

分析揭示了该领域的清晰演进轨迹：

- **PPO**（2017 年）建立了裁剪代理基础和双网络架构，支撑了第一代对齐 LLM（InstructGPT、ChatGPT）。
- **DPO**（2023 年）证明了通过消除显式奖励建模可以大幅简化 RLHF 流程，以在线探索换取离线简洁性。
- **GRPO**（2025 年）通过组优势归一化消除了价值网络，内存减少 ∼50%，简化了可验证推理任务的架构。
- **DAPO**（2025 年）通过三项针对性创新扩展 GRPO——非对称 Clip-Higher、动态采样和 Token 级损失——在 AIME 2024 上实现 67% 的相对提升。

对实践者的建议明确：对于有偏好数据的主观对齐任务使用 **DPO**，对于资源受限的高效推理使用 **GRPO**，对于追求最大推理性能使用 **DAPO**，当需要学习型奖励模型和理论保证时使用 **PPO**。该领域正在快速发展，混合方法和过程级反馈代表了特别有前景的未来研究方向。

参考文献

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. ICLR*, 2016.

[3] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. NeurIPS*, 2023.

[4] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[5] Q. Yu *et al.*, "DAPO: An open-source LLM reinforcement learning system at scale," *arXiv preprint arXiv:2503.14476*, 2025.

[6] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Proc. NeurIPS*, 2022.

[7] P. F. Christiano *et al.*, "Deep reinforcement learning from human preferences," in *Proc. NeurIPS*, 2017.

[8] D. M. Ziegler *et al.*, "Fine-tuning language models from human preferences," *arXiv preprint arXiv:1909.08593*, 2019.

[9] N. Stiennon *et al.*, "Learning to summarize with human feedback," in *Proc. NeurIPS*, 2020.

[10] Y. Bai *et al.*, "Training a helpful and harmless assistant with reinforcement learning from human feedback," *arXiv preprint arXiv:2204.05862*, 2022.

[11] H. Touvron *et al.*, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[12] J. Achiam *et al.*, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[13] L. Zheng *et al.*, "Secrets of RLHF in large language models part I: PPO," *arXiv preprint arXiv:2307.04964*, 2023.

[14] W. Yuan *et al.*, "Self-rewarding language models," *arXiv preprint arXiv:2401.10020*, 2024.

[15] G. Cui *et al.*, "UltraFeedback: Boosting language models with scaled AI feedback," *arXiv preprint arXiv:2310.01377*, 2024.

[16] Z. Wang *et al.*, "HelpSteer: Multi-attribute helpfulness dataset for SteerLM," *arXiv preprint arXiv:2311.09528*, 2024.

[17] Y. Meng *et al.*, "SimPO: Simple preference optimization with reference-free reward," *arXiv preprint arXiv:2405.14734*, 2024.

[18] J. Hong *et al.*, "ORPO: Monolithic preference optimization without reference model," *arXiv preprint arXiv:2403.07691*, 2024.

[19] M. G. Azar *et al.*, "A general theoretical paradigm to understand learning from human preferences," in *Proc. ICML*, 2024.

[20] C. Rosset *et al.*, "Direct Nash optimization: Teaching language models to self-improve more effectively," *arXiv preprint arXiv:2404.03715*, 2024.

[21] Z. Guo *et al.*, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," *arXiv preprint arXiv:2402.03300*, 2024.

[22] Qwen Team, "Qwen2 technical report," *arXiv preprint arXiv:2407.10671*, 2024.

[23] A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[24] A. Dubey *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[25] C. Snell *et al.*, "Scaling LLM test-time compute optimally can be more effective than scaling model parameters," *arXiv preprint arXiv:2408.03314*, 2024.

[26] H. Lightman *et al.*, "Let's verify step by step," in *Proc. ICLR*, 2024.

[27] J. Uesato *et al.*, "Solving math word problems with process and outcome-based feedback," in *Proc. NeurIPS*, 2022.

[28] P. Wang *et al.*, "Math-Shepherd: Verify and reinforce LLMs step-by-step," *arXiv preprint arXiv:2312.08935*, 2024.

[29] A. Zeng *et al.*, "GLM-4 technical report," *arXiv preprint arXiv:2406.12793*, 2024.